ShopSense

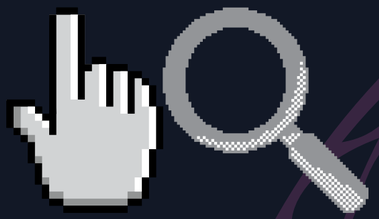
An AI-Powered Online Store Database

Start

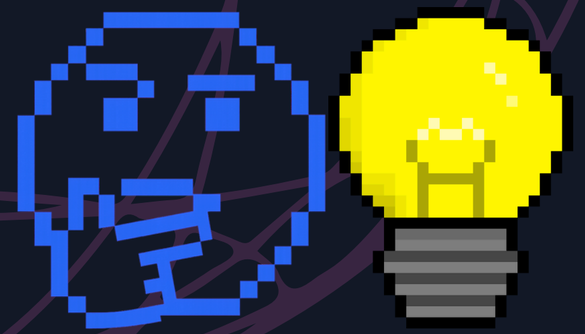


Member 1 - Madhur Grover
Member 2 - Arjun Atheesh

PROBLEM



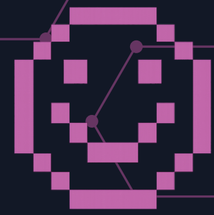
- Small online sellers run their stores on spreadsheets or basic tools, with no real database behind them
- Overselling: two customers buy the last item at the same moment
- Weak security: plain-text passwords, and anyone can reach admin functions
- Poor discovery: keyword search fails ("warm winter wear" finds nothing)
- No insights: sellers can't see what sells, what's running out, or what customers think



SOLUTION 

- A desktop app built in Python on a well-designed PostgreSQL database
- Customers browse, search, buy and track orders
- Admins manage products and orders and see analytics
- AI built into the shopping flow itself, not added as a separate chatbot
- Every action (signup, cart, order, review) is stored with data integrity guaranteed

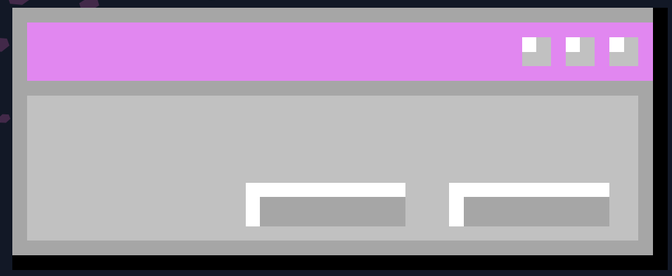
‘One secure database, two roles, AI in the flow’



TECH STACK

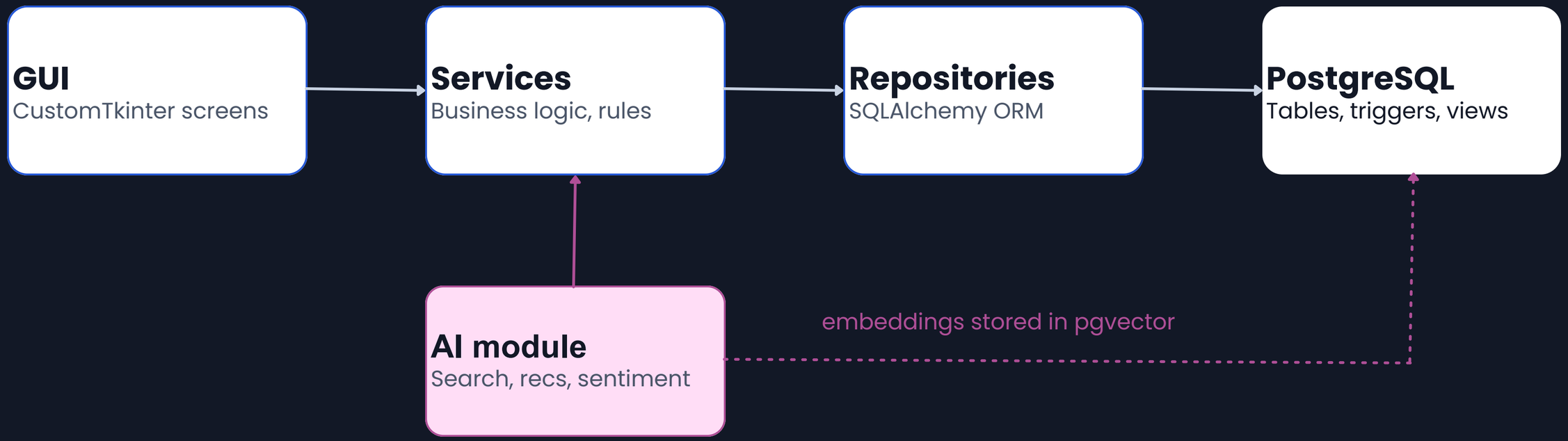


- Language: Python 3.11
- Database: PostgreSQL (hosted on Neon) with the pgvector extension
- ORM: SQLAlchemy, with Alembic for migrations
- GUI: CustomTkinter, with matplotlib for charts
- Security: bcrypt password hashing
- AI/ML: Sentence-Transformers (MiniLM), VADER, scikit-learn
- Tools: GitHub, pytest, Faker for test data



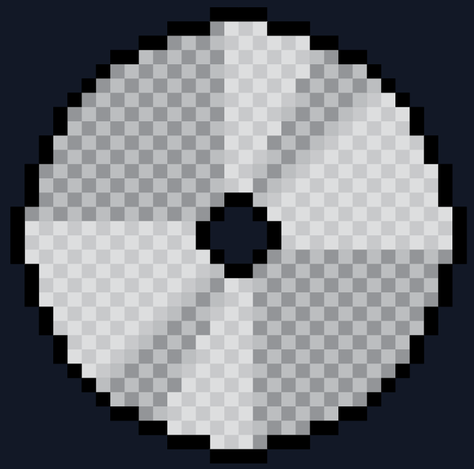
SYSTEM ARCHITECTURE

Four layers, each with one job. The GUI never talks to the database directly.



KEY FEATURES

- Registration and login with role-based access (admin vs customer)
- Product management: add, update and delete products
- Shopping cart, checkout, coupons and mock payment records
- Order tracking with a status timeline (Placed → Shipped → Delivered)
- Order history reports exportable as CSV or PDF
- Admin dashboard with KPIs and charts

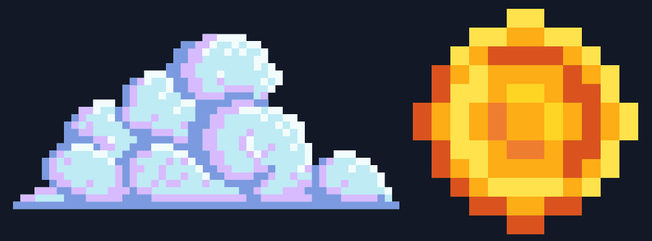


UNIQUE FEATURES

- Oversell-proof checkout using database-level row locking
- Security first: bcrypt hashing, account lockout after failed logins, audit log of admin actions
- Smart dashboard: revenue trends, best sellers and low-stock alerts
- Realistic seed data (200 products, 500 orders), so the analytics show real patterns



THE AI FEATURES



- Semantic search: queries match by meaning, not exact keywords
- Recommendations: "frequently bought together" from products in the same orders; "similar products" from embedding similarity
- Review sentiment: Scores the reviews and the negative reviews are flagged so admin can see
- Stretch goals: ask-your-data (plain English → read-only SQL via an LLM) and low-stock forecasting

THANK YOU

GitHub: [repo link]

